```
#Task 1
import random

class CasinoAgent:
    def __init__(self, n):
        self.n = n
        self.players = [f"Player {i+1}" for i in range(n)]
        self.cards = self.generate_cards(n)
        self.assigned = {}
        self.used_players = set()
        self.used_cards = set()

    def generate_cards(self, n):
        suits = ["Spades", "Hearts", "Diamonds", "Clubs"]
        cards = []

        for i in range(n):
            value = random.randint(1, 13)
            suit = random.choice(suits)
            cards.append((value, suit))
        return cards

    def roll_dice(self):
        player_roll = random.randint(0, self.n - 1)
        card_roll = random.randint(0, self.n - 1)
        return player_roll, card_roll

    def assign_cards(self):
        while len(self.assigned) < self.n:
            p, c = self.roll_dice()

            if p not in self.used_players and c not in self.used_cards:
                player = self.players[p]
                card = self.cards[c]

                self.assigned[player] = card
                self.used_players.add(p)
                self.used_cards.add(c)

                print(f"{player} gets card {card}")

    def card_priority(self, card):
        value, suit = card
        suit_priority = {
            "Spades": 4,
            "Hearts": 3,
            "Diamonds": 2,
            "Clubs": 1
        }
        return (value, suit_priority[suit])

    def find_winner(self):
        winner = max(self.assigned, key=lambda p: self.card_priority(self.assigned[p]))
        return winner, self.assigned[winner]


n = int(input("Enter number of players: "))
agent = CasinoAgent(n)

agent.assign_cards()

winner, card = agent.find_winner()
print(f"\nWinner is {winner} with card {card}")
```

```
Enter number of players: 5
Player 2 gets card (12, 'Hearts')
Player 1 gets card (6, 'Spades')
Player 3 gets card (12, 'Diamonds')
Player 4 gets card (1, 'Spades')
Player 5 gets card (5, 'Diamonds')

Winner is Player 2 with card (12, 'Hearts')
```

```
#Task 2
class MultiAgentSystem:
```

```
    #Goal-Based Agent
    class GoalBasedAgent:
        def __init__(self, start, goal):
            self.position = start
            self.goal = goal

        def step(self):
            x, y = self.position
            gx, gy = self.goal

            if x < gx:
                self.position = (x + 1, y)
            elif x > gx:
                self.position = (x - 1, y)
            elif y < gy:
                self.position = (x, y + 1)
            elif y > gy:
                self.position = (x, y - 1)

            return self.position

        def run(self):
            print("\n Goal-Based Agent")
            while self.position != self.goal:
                print("Position:", self.position)
                self.step()
            print("Position:", self.position)
            print("Goal Reached!")

    #Model-Based Agent
    class ModelBasedAgent:
        def __init__(self):
            self.environment = ['Dirty', 'Dirty', 'Dirty']
            self.location = 0

        def perceive(self):
            return self.environment[self.location]

        def act(self):
            if self.perceive() == 'Dirty':
                print(f"Cleaning location {self.location}")
                self.environment[self.location] = 'Clean'
            else:
                print(f"Moving from {self.location}")
                self.location = (self.location + 1) % len(self.environment)

        def run(self):
            print("\n Model-Based Agent ")
            for _ in range(6):
                self.act()

    #Utility-Based Agent
    class UtilityBasedAgent:
        def __init__(self):
            self.options = {
                "Option A": 40,
                "Option B": 90,
                "Option C": 70
            }

        def choose(self):
            best = max(self.options, key=self.options.get)
            print("\n Utility-Based Agent")
            print(f"Best choice: {best} with utility {self.options[best]}")



#Run All Agents
system = MultiAgentSystem()

goal_agent = system.GoalBasedAgent((0, 0), (3, 3))
model_agent = system.ModelBasedAgent()
utility_agent = system.UtilityBasedAgent()

goal_agent.run()
model_agent.run()
utility_agent.choose()
```

```
 Goal-Based Agent
Position: (0, 0)
Position: (1, 0)
Position: (2, 0)
Position: (3, 0)
Position: (3, 1)
Position: (3, 2)
Position: (3, 3)
Goal Reached!

 Model-Based Agent
Cleaning location 0
Moving from 0
Cleaning location 1
Moving from 1
Cleaning location 2
Moving from 2

 Utility-Based Agent
Best choice: Option B with utility 90
```